AI RESPONSE QUALITY EVALUATOR AGENT

Model Research, System Architecture & Workflow

1. Executive Summary

The AI Response Quality Evaluator Agent is a proposed platform for checking answers produced by AI systems such as chatbots, virtual assistants, and coding assistants. Instead of relying only on manual review, it uses four specialized judge agents to assess relevance, factual accuracy, hallucination risk, and completeness. A Verdict Agent combines their results into an overall score, a verdict, and an explanation.

The platform supports one response at a time as well as CSV-based batch evaluation. When source documents are available, a Retrieval-Augmented Generation (RAG) pipeline retrieves useful passages so judges can compare claims against evidence. Results are presented in a dashboard and can be exported as a PDF report.

2. Problem Statement

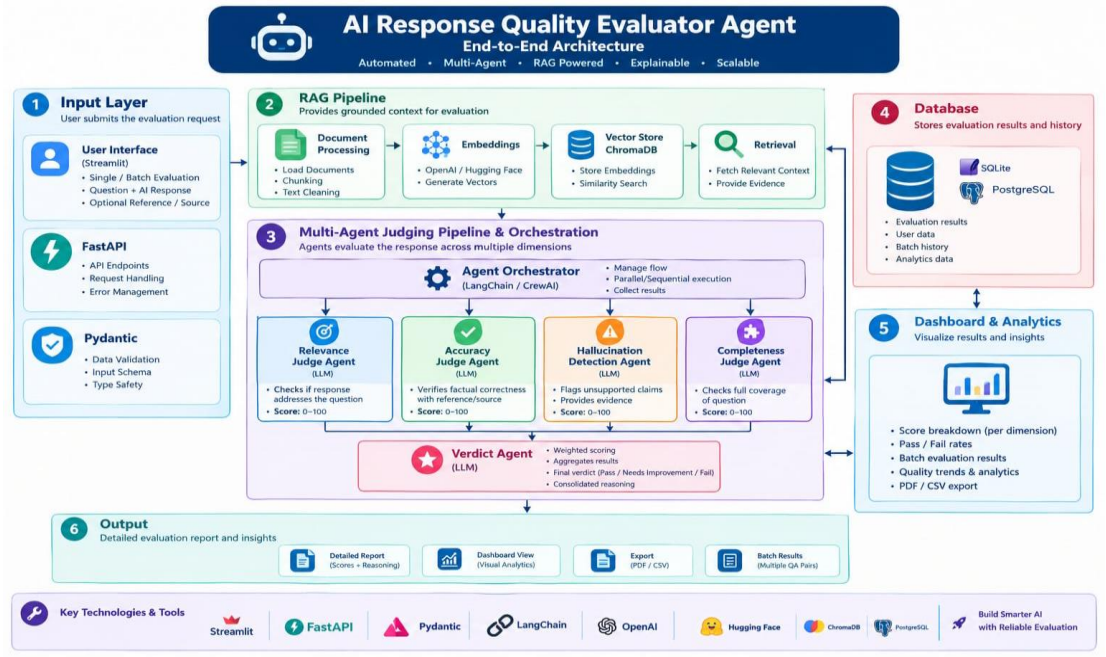
Manual evaluation can be slow, inconsistent, and difficult to scale. AI answers may be off-topic, factually wrong, incomplete, or contain claims that are not supported by the available evidence. The project aims to make this review process more repeatable, transparent, and easier to analyze.

3. Project Objectives

- Evaluate responses across relevance, accuracy, hallucination risk, and completeness.
- Show a score and a plain-language reason for each evaluation dimension.
- Use a reference answer or passages retrieved from source documents where available.
- Support both single-response evaluation and CSV-based batch processing.
- Combine individual judge results into Pass, Needs Improvement, or Fail.
- Visualize scores, pass/fail rates, unsupported-claim flags, and quality trends.
- Export evaluation results and explanations in a PDF report.

4. System Architecture

The architecture is organized as a sequence of connected layers. The interface collects the evaluation input, the backend validates it, the RAG pipeline finds evidence when documents are provided, judge agents evaluate the answer, and the Verdict Agent combines the findings. The database and dashboard then store and present the results.



4.1 Architecture layers in simple language

Layer	What it does
Input layer	The user submits a question and the AI-generated answer. A reference answer or source document is optional.
Validation and API	Pydantic checks input fields and data types. FastAPI can provide API endpoints; Streamlit can provide the user interface.
RAG evidence pipeline	If documents are supplied, text is extracted, split into chunks, converted into embeddings, indexed in ChromaDB or FAISS, and retrieved when relevant.
Judge-agent layer	Four specialized agents independently evaluate relevance, accuracy, hallucination risk, and completeness.
Verdict and scoring	The Verdict Agent combines judge outputs, applies an agreed scoring policy, and explains the overall result.
Data and reporting	SQLite can store evaluation history. A dashboard can display detailed scores, batch summaries, trends, and report exports.

5. Agent Responsibilities

Agent	Responsibility
Relevance Judge Agent	Checks how directly the answer responds to the question. Returns a score and explanation.
Accuracy Judge Agent	Checks factual claims against the supplied reference answer or retrieved source evidence and reports discrepancies.
Hallucination Detection Agent	Flags claims that are unsupported or contradicted by the reference and identifies relevant evidence. An unsupported claim is not automatically false.
Completeness Judge Agent	Checks whether all parts of the question have been answered and identifies omissions.
Verdict Agent	Combines the four judge outputs into an overall score, verdict, and consolidated explanation.

6. End-to-End Workflow

1. Submit	User enters the question and AI response, with an optional reference answer or document.
2. Validate	The application checks required fields, types, and input format.
3. Prepare evidence	If a document is provided, extract text, split it into chunks, create embeddings, and index it for retrieval.
4. Retrieve	Find passages relevant to the question or claims; a short reference answer can be compared directly.
5. Judge	The four judge agents assess relevance, accuracy, hallucination risk, and completeness.
6. Combine	The Verdict Agent aggregates scores and explanations using the agreed scoring rules.
7. Display and save	Show per-dimension results and the overall verdict; store evaluation history.
8. Analyze and export	The dashboard summarizes trends and batch results; users can export reports.

7. Proposed Scoring Method

The supplied project summary proposes scoring each dimension from 0 to 10 and gives an initial weighting example. These weights and verdict thresholds are proposals and should be confirmed with the project mentor before implementation.

Dimension	Symbol	Example weight
Relevance	**R**	25%
Accuracy	**A**	35%
Hallucination Safety	**H**	25%
Completeness	**C**	15%

Overall Score = 0.25R + 0.35A + 0.25H + 0.15C

Important: Define the direction of the Hallucination Safety score consistently (a higher score should mean safer/fewer unsupported claims). The example thresholds shown in the source summary are provisional and require mentor approval. Serious factual errors should not be hidden by a high average.

8. Proposed Technology Stack

Technology	Purpose
Python	Core implementation and evaluation logic
LLM API	Reasoning for judge agents
Streamlit	Input interface and results dashboard
Pandas	CSV and batch processing
Sentence Transformers	Text embeddings
ChromaDB or FAISS	Vector storage and retrieval
SQLite (optional)	Evaluation history
Plotly	Charts and analytics
PyMuPDF	PDF text extraction
ReportLab	PDF report generation
FastAPI (optional)	Separate backend and API endpoints

9. Testing and Validation

- Test relevant and irrelevant answers, correct and incorrect claims, unsupported claims, and incomplete answers.
- Compare a sample of agent judgments with human-reviewed examples.
- Check that flagged claims include relevant evidence and distinguish unsupported claims from contradicted claims.
- Test CSV input validation, row-level errors, score aggregation, dashboard calculations, and PDF export.
- Review consistency of scores and explanations across varied questions and source documents.

10. Eight-Week Implementation Roadmap

Timeline	Planned work
Weeks 1–2	Research evaluation and RAG; finalize architecture; build input interface and reference knowledge base.
Weeks 3–4	Implement Relevance, Accuracy, and Hallucination Detection agents; test varied question-answer pairs.
Weeks 5–6	Implement Completeness and Verdict agents; integrate scoring display and CSV batch evaluation.
Weeks 7–8	Build dashboard and PDF export; perform end-to-end testing; complete documentation and final demo.

11. Expected Deliverables

- Single-response evaluation interface.
- Four judge agents and one Verdict Agent.
- Reference knowledge base and RAG retrieval pipeline.
- CSV batch evaluation and results management.

- Scoring dashboard with quality analytics.
- PDF evaluation report.
- Test results, technical documentation, project report, and final demonstration.

12. Key Design Notes

- RAG is conditional: use it when source documents or a knowledge base are available. A short reference answer can be compared directly.
- Make judge outputs structured so scores, reasons, evidence, and omissions can be saved and displayed consistently.
- Keep the proposed weights and verdict thresholds configurable until reviewed by the mentor.
- Treat unsupported claims carefully: lack of evidence is not the same as proof that a claim is false.
- Clearly label the architecture and technology choices as proposed until implementation and testing confirm them.

13. Conclusion

The proposed AI Response Quality Evaluator Agent provides a structured way to evaluate AI answers using multiple specialized judges, reference evidence, explainable scoring, and analytics. The architecture separates input handling, evidence retrieval, judging, verdict generation, storage, and reporting so the project can be implemented and tested in stages.

.